
Weekly Report: [W08]

General Information

- **Group ID:** 52
- **Group Name:** Group 52
- **Class:** 25A01
- **Project Name:** Super Mario Game
- **Date range:** 2026-07-26 – 2026-08-01
- **Github Repository:** <https://github.com/ndmhuy/SuperMarioGame.git>

Tasks Completed This Week

25125083 - Nguyễn Đình Minh Huy (Member A - Engine, Physics & Player/Item Entities)

Branch: `dev`

- **Task 1: ParticleSystem Architecture & ObjectPool Design Review:** Reviewed and validated Member B's `ParticleSystem` implementation using the Object Pool pattern (`ObjectPool<Particle>`). Verified zero-allocation particle recycling design achieves O(1) acquire/release with a fixed-capacity pool, avoiding heap fragmentation during gameplay bursts (coin sparkles, brick debris, fireball trails).
 - **Task 2: TASKS.md Matrix Updates:** Updated the sequential task checklist `TASKS.md` to tick off all completed phases and subtasks through Phase 8 (Save/Load & Persistence) and Bonus A (Level Editor). Cross-referenced task completion status with `TASK_DIVISION.md` to ensure parity between division matrix and checklist.
 - **Task 3: Repository Synchronization & Remote Branch Tracking:** Executed `git fetch --all` to retrieve all remote branches. Verified local `dev` branch is up to date with `origin/dev` and tracked the status of `B/feature/graphic-visual` for Member B's particle and sprite sheet work.
-

25125084 - Trần Gia Huy (Member B - Input, Sound & Gameplay Systems)

Branch: `B/feature/graphic-visual`

- **Task 1: ParticleSystem Implementation with ObjectPool:** Implemented `ParticleSystem` with integrated `ObjectPool<Particle>` for zero-allocation particle lifecycle management. Supports burst and continuous emission modes with configurable velocity, lifetime, color, and gravity parameters. Commit: `7a6f62b`.
- **Task 2: Particle Effect Tuning & Configuration:** Tuned particle effect parameters for improved visibility and visual fidelity across coin collection sparkles, brick block debris, and power-up activation bursts. Adjusted alpha fade curves, emission rates, and spread angles. Commits: `c7d4fd4`, `62c78ef`.
- **Task 3: Particle Visual Test Runner:** Created `verify_particles_visual` interactive test target for real-time particle effect debugging with ImGui sliders for emission rate, lifetime, velocity, gravity, and color channels.
- **Task 4: Player Animation Spritesheet Extraction:** Created player character sprite sheet `player.json` with 92 animation frames extracted from atlas sources. Established frame naming conventions for idle, walk, run, jump, crouch, slide, swim, climb, and power-up states. Commits: `d6bd07d`, `0be4508`, `9403be5`.
- **Task 5: Player Animation Visual Test Runner:** Created `verify_player_animation_visual` interactive test target for previewing all player animation state transitions with real-time sprite sheet playback controls.
- **Task 6: Asset Pipeline Scripts & Sprite Naming:** Began sprite naming standardization (`chore, wip: naming sprites`). Established naming conventions for sprite atlas frame identifiers. Commit: `15327b4`.
- **Task 7: Minimap Overlay Component:** Implemented `Minimap` component with scaled tilemap preview, camera view frustum indicator, and real-time player tracking dot overlay. Created `verify_minimap_visual` test target. Commit: `cb6eab2`.
- **Task 8: Random Function Improvement:** Enhanced random number generation utility with uniform real distribution for particle spread calculations. Commit: `9bb9d16`.

AI Usage Declaration

- Used AI to review ObjectPool zero-allocation particle recycling design and validate O(1) acquire/release guarantees for fixed-capacity pool management.
- Used AI to extract player sprite atlas coordinate boundaries from source sprite sheets and generate JSON frame descriptor files.

-
- Used AI to design uniform real random distribution seeding for particle velocity spread angle calculations.
-

Tasks Planned for Next Week

- **Nguyễn Đình Minh Huy:** Implement Fireball entity with ObjectPool, Flagpole scoring system, Trampoline bounce mechanics, CheckpointFlag entity, and Memento pattern (TimeRewindManager circular buffer). Begin MapGenerator for procedural level generation.
 - **Trần Gia Huy:** Complete spritesheet merging tools for enemy_projectile, item, world_scenery sprite sheets. Implement SpriteColorFilter, SpriteTransformAnim, and EntityDeathEffect graphics subsystems.
-

Issues & Resolutions

Issue 1: ObjectPool Zero-Allocation Particle Recycling

- **Problem:** Naive particle systems allocate and deallocate `Particle` objects on every emission burst, causing heap fragmentation and GC-like pauses during high-density particle events (e.g., 50+ coin sparkles simultaneously).
- **Resolution:** Implemented `ObjectPool<Particle>` with a pre-allocated fixed-capacity vector. `acquire()` returns a pointer to an inactive pooled particle (O(1) linear scan of active flags), and `release()` marks the particle inactive without deallocation. Pool capacity is set at initialization and never grows, ensuring zero heap allocations during gameplay.

Issue 2: Player Sprite Atlas Coordinate Boundary Extraction

- **Problem:** Source sprite sheets contained variable-sized frames with inconsistent padding, making automated frame boundary detection unreliable for generating JSON frame descriptors.
- **Resolution:** Implemented boundary detection scanning for non-transparent pixel regions within atlas grid cells, with configurable padding tolerance thresholds. Generated `player.json` with 92 validated frame rectangles mapped to animation state identifiers.

Issue 3: Uniform Real Random Distribution for Particle Spread

- **Problem:** Using `rand() % range` for particle velocity spread angles produced non-uniform distributions with visible clustering artifacts in radial emission patterns.
- **Resolution:** Replaced with C++17 `std::uniform_real_distribution<float>` seeded from `std::random_device` via `std::mt19937`, producing visually uniform radial particle spread patterns.